BASES DE DATOS

LOVEC<3DE

30% 3º EVALUACIÓN

PABLO PEDRÓN LÓPEZ

1º DAW CURSO 2025-2026

26/04/2026

ÍNDICE

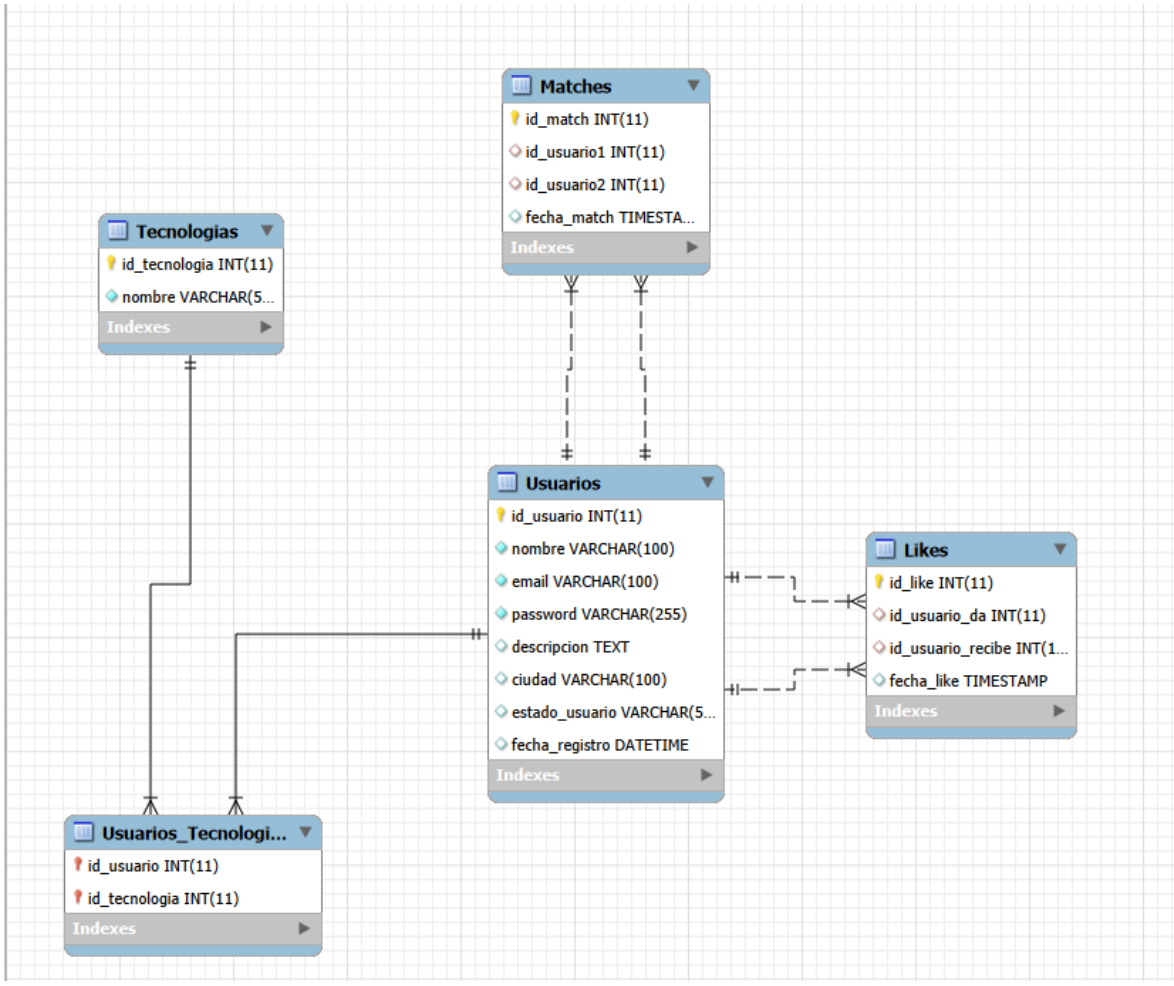
DISEÑO ESTRUCTURAL 3
DIAGRAMA ENTIDAD-RELACIÓN 3
USUARIOS 4
PROCEDIMIENTOS ALMACENADOS 5

DISEÑO ESTRUCTURAL

La base de datos LoveCode es una plataforma de "dating" para programadores. El modelo se centra en conectar usuarios según sus afinidades tecnológicas.

- Usuarios: Almacena el perfil, descripción y ubicación.
- Tecnologías: Un catálogo de lenguajes y herramientas (Java, Python, etc.).
- Usuarios_Tecnologias: Relación N:M que define el stack de cada programador.
- Likes: Registra el interés unidireccional entre usuarios.
- Matches: Tabla que formaliza la conexión mutua.

DIAGRAMA ENTIDAD-RELACIÓN



USUARIOS

```
MariaDB [(none)]> create database lovecode;
Query OK, 1 row affected (0,001 sec)

MariaDB [(none)]> use lovecode;
Database changed
MariaDB [lovecode]> CREATE USER 'administrador'@'localhost' IDENTIFIED BY 'administradorpablo';
Query OK, 0 rows affected (0,008 sec)

MariaDB [lovecode]> GRANT ALL PRIVILEGES ON LoveCode.* TO 'administrador'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0,006 sec)

MariaDB [lovecode]> CREATE USER 'desarrollador'@'localhost' IDENTIFIED BY 'desarrolladorpablo';
Query OK, 0 rows affected (0,003 sec)

MariaDB [lovecode]> GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, INDEX, REFERENCES ON LoveCode.* TO 'desarrollador'@'localhost';
Query OK, 0 rows affected (0,002 sec)

MariaDB [lovecode]> CREATE USER 'lector'@'localhost' IDENTIFIED BY 'lectorpablo';
Query OK, 0 rows affected (0,004 sec)

MariaDB [lovecode]> GRANT SELECT ON LoveCode.* TO 'lector'@'localhost';
Query OK, 0 rows affected (0,005 sec)

MariaDB [lovecode]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)
```

En el entorno de sistemas de LoveCode, se ha implementado una estructura de control de acceso basada en tres perfiles con distintos niveles de responsabilidad para garantizar la seguridad de la base de datos:

Usuario Administrador (**administrador**): Es el perfil con mayores privilegios (ALL PRIVILEGES) sobre la base de datos lovecode_db. Este usuario ‘root’ tiene todo el control.

Usuario Desarrollador (**desarrollador**): Este perfil está orientado a la fase de construcción y pruebas del backend. Tiene permisos para insertar, actualizar y consultar datos (DML).

Usuario Lector (**lector**): Es un perfil restringido únicamente a la consulta de información (SELECT). Se utiliza para tareas de auditoría o visualización simple, asegurando que bajo ninguna circunstancia se puedan modificar o eliminar los registros existentes de usuarios o tecnologías en el sistema.

Posteriormente se ha editado el “localhost” por “%” para que se puedan utilizar en una máquina remota que no sea la propia de debian donde esta alojada la BBDD.

PROCEDIMIENTOS ALMACENADOS

Borrar usuarios

Query 1 | borrar_usuario - Routine

Name: borrar_usuario

The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
CREATE DEFINER=`administrador`@`%` PROCEDURE `borrar_usuario`(IN p_id_usuario INT)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Usuarios WHERE id_usuario = p_id_usuario) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: el usuario no existe';
    ELSE
        DELETE FROM Usuarios WHERE id_usuario = p_id_usuario;
        SELECT 'Usuario eliminado correctamente' AS mensaje;
    END IF;
END
```

Si no existe te envia un mensaje de error, sin el usuario existe lo borra y te da un mensaje de usuario borrado.

Contar matches

Query 1 | borrar_usuario - Routine | contar_matches - Routine

Name: contar_matches

The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
CREATE DEFINER=`administrador`@`%` PROCEDURE `contar_matches`(IN p_id_usuario INT)
BEGIN
    SELECT COUNT(*) AS total_matches
    FROM Matches
    WHERE id_usuario1 = p_id_usuario
       OR id_usuario2 = p_id_usuario;
END
```

Hace un select de matches de un usuario y los cuenta sacando una tabla independientemente si esta en el “lado” 1 o 2.

Crear usuario

Query 1 | borrar_usuario - Routine | contar_matches - Routine | crear_usuario - Routine

Name: crear_usuario

The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
CREATE DEFINER=`administrador`@`%` PROCEDURE `crear_usuario`(
    IN p_nombre       VARCHAR(100),
    IN p_email        VARCHAR(100),
    IN p_password     VARCHAR(255),
    IN p_descripcion  TEXT,
    IN p_ciudad       VARCHAR(100),
    IN p_estado       VARCHAR(50)
)
BEGIN
    IF EXISTS (SELECT 1 FROM Usuarios WHERE email = p_email) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: el email ya está registrado';
    ELSE
        INSERT INTO Usuarios (nombre, email, password, descripcion, ciudad, estado_usuario)
        VALUES (p_nombre, p_email, p_password, p_descripcion, p_ciudad, p_estado);
        SELECT LAST_INSERT_ID() AS id_usuario_creado;
    END IF;
END
```

Se nombran los atributos con tipos de datos, si existe usuario te lanza un mensaje, si no lo inserta y te hace un select de el ultimo usuario añadido, es decir el que acabas de crear.u